

A Detailed Explanation of Variational Autoencoders (VAEs)

Generated Explanation

April 19, 2025

1 The Goal: Generative Modeling

The primary goal of a Variational Autoencoder (VAE) is to learn a generative model for some data $X = \{\mathbf{x}^{(i)}\}_{i=1}^N$. We want to be able to generate new data points $\mathbf{x}_{new}$ that look like they came from the same underlying distribution $p_{data}(\mathbf{x})$ as the training data.

We approach this by introducing a latent variable $\mathbf{z}$ (usually lower-dimensional than $\mathbf{x}$). We assume that our data $\mathbf{x}$ is generated from this latent variable $\mathbf{z}$ via some process. This involves two distributions:

- **Prior distribution over latent variables:** $p(\mathbf{z})$. This represents our belief about the structure of the latent space *before* seeing any data. It's typically chosen to be simple, often a standard multivariate Gaussian:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}) \quad (1)$$

where $\mathbf{0}$ is the zero vector and $\mathbf{I}$ is the identity matrix. This means each component z_j of the latent vector $\mathbf{z}$ is independently drawn from a standard normal distribution $\mathcal{N}(0, 1)$. The probability density function (PDF) for a D_z -dimensional latent space is:

$$p(\mathbf{z}) = \prod_{j=1}^{D_z} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}z_j^2\right) = \frac{1}{(2\pi)^{D_z/2}} \exp\left(-\frac{1}{2}\mathbf{z}^T \mathbf{z}\right) \quad (2)$$

- **Likelihood (or Decoder) distribution:** $p_\theta(\mathbf{x}|\mathbf{z})$. This describes the probability of generating data point $\mathbf{x}$ given a specific latent variable $\mathbf{z}$. This distribution is parameterized by θ , typically the weights and biases of a neural network (the decoder). The form of this distribution depends on the type of data $\mathbf{x}$ (e.g., Bernoulli for binary images, Gaussian for real-valued data). We'll detail this later.

2 The Problem: Intractable Marginal Likelihood and Posterior

Our ultimate goal is to maximize the probability of observing our dataset X . Assuming the data points are i.i.d., we want to maximize the marginal likelihood of each data point $\mathbf{x}$:

$$p(\mathbf{x}) = \int p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (3)$$

This integral is generally intractable because it involves integrating over all possible latent variables $\mathbf{z}$, which is often a high-dimensional space.

Furthermore, during learning and for some applications, we might need the true posterior distribution $p_\theta(\mathbf{z}|\mathbf{x})$, which tells us the probability of the latent variables *given* an observed data point $\mathbf{x}$. Using Bayes' theorem:

$$p_\theta(\mathbf{z}|\mathbf{x}) = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})} = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{\int p_\theta(\mathbf{x}|\mathbf{z}')p(\mathbf{z}')d\mathbf{z}'} \quad (4)$$

This is also intractable because the denominator $p(\mathbf{x})$ is intractable.

3 The Solution: Variational Inference

Since we can't compute or optimize $p(\mathbf{x})$ or $p_\theta(\mathbf{z}|\mathbf{x})$ directly, we introduce an *approximate* posterior distribution $q_\phi(\mathbf{z}|\mathbf{x})$. This distribution is designed to be tractable and is parameterized by ϕ , typically the weights and biases of another neural network (the encoder). The goal is to make $q_\phi(\mathbf{z}|\mathbf{x})$ as close as possible to the true posterior $p_\theta(\mathbf{z}|\mathbf{x})$.

A common choice for $q_\phi(\mathbf{z}|\mathbf{x})$ is a multivariate Gaussian distribution with a diagonal covariance matrix:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))) \quad (5)$$

Here:

- $\boldsymbol{\mu}_\phi(\mathbf{x})$ is the mean vector.
- $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$ is the variance vector (diagonal elements of the covariance matrix).
- The encoder network takes $\mathbf{x}$ as input and outputs the parameters $\boldsymbol{\mu}_\phi(\mathbf{x})$ and $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$ (or often $\log \boldsymbol{\sigma}_\phi^2(\mathbf{x})$ for numerical stability).

The **explicit formula** for the probability density function (PDF) of $q_\phi(\mathbf{z}|\mathbf{x})$, assuming a D_z -dimensional latent space, is:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \prod_{j=1}^{D_z} \frac{1}{\sqrt{2\pi\sigma_{j,\phi}^2(\mathbf{x})}} \exp\left(-\frac{(z_j - \mu_{j,\phi}(\mathbf{x}))^2}{2\sigma_{j,\phi}^2(\mathbf{x})}\right) \quad (6)$$

$$= \frac{1}{(2\pi)^{D_z/2} \prod_{j=1}^{D_z} \sigma_{j,\phi}(\mathbf{x})} \exp\left(-\frac{1}{2} \sum_{j=1}^{D_z} \frac{(z_j - \mu_{j,\phi}(\mathbf{x}))^2}{\sigma_{j,\phi}^2(\mathbf{x})}\right) \quad (7)$$

In vector notation using the diagonal covariance matrix $\boldsymbol{\Sigma}_\phi(\mathbf{x}) = \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))$:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \frac{1}{(2\pi)^{D_z/2} |\boldsymbol{\Sigma}_\phi(\mathbf{x})|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{z} - \boldsymbol{\mu}_\phi(\mathbf{x}))^T \boldsymbol{\Sigma}_\phi(\mathbf{x})^{-1} (\mathbf{z} - \boldsymbol{\mu}_\phi(\mathbf{x}))\right) \quad (8)$$

where $|\boldsymbol{\Sigma}_\phi(\mathbf{x})|$ is the determinant of the covariance matrix, which simplifies to $\prod_{j=1}^{D_z} \sigma_{j,\phi}^2(\mathbf{x})$.

4 The Objective Function: The Evidence Lower Bound (ELBO)

We want to maximize the marginal log-likelihood $\log p(\mathbf{x})$. We can derive a lower bound on this quantity using our approximate posterior $q_\phi(\mathbf{z}|\mathbf{x})$.

Start with the definition of the Kullback-Leibler (KL) divergence between the approximate posterior and the true posterior:

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) = \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z}|\mathbf{x})} d\mathbf{z} \quad (9)$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p_\theta(\mathbf{z}|\mathbf{x})] \quad (10)$$

Now substitute $p_\theta(\mathbf{z}|\mathbf{x}) = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}$:

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p_\theta(\mathbf{x}|\mathbf{z}) - \log p(\mathbf{z}) + \log p(\mathbf{x})] \quad (11)$$

Since $\log p(\mathbf{x})$ does not depend on $\mathbf{z}$, the expectation is just $\log p(\mathbf{x})$:

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})] - \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] + \log p(\mathbf{x}) \quad (12)$$

The first term is the KL divergence between the approximate posterior and the prior:

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})] \quad (13)$$

So we have:

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) = D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) - \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] + \log p(\mathbf{x}) \quad (14)$$

Rearranging for $\log p(\mathbf{x})$:

$$\log p(\mathbf{x}) = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) + D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x})) \quad (15)$$

Since KL divergence is always non-negative ($D_{\text{KL}}(q || p) \geq 0$), we have:

$$\log p(\mathbf{x}) \geq \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) \quad (16)$$

This right-hand side is the **Evidence Lower Bound (ELBO)**, often denoted $\mathcal{L}(\theta, \phi; \mathbf{x})$:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \underbrace{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction Term}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))}_{\text{KL Divergence / Regularization Term}} \quad (17)$$

Maximizing the ELBO with respect to parameters θ and ϕ serves two purposes:

1. It pushes $\log p(\mathbf{x})$ higher (our original goal).
2. It minimizes the KL divergence between the approximate posterior $q_\phi(\mathbf{z}|\mathbf{x})$ and the true posterior $p_\theta(\mathbf{z}|\mathbf{x})$ (because $\log p(\mathbf{x})$ is fixed for a given $\mathbf{x}$, and the ELBO is $\log p(\mathbf{x}) - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z}|\mathbf{x}))$).

5 Expanding the ELBO Terms

Let's look at the two terms in the ELBO (Equation 17) in more detail.

5.1 KL Divergence Term

This term measures how much the approximate posterior (learned by the encoder) deviates from the prior $p(\mathbf{z})$. It acts as a regularizer, encouraging the encodings to fill the space around the origin like a standard Gaussian. When $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x})))$ and $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$, this KL divergence has a convenient analytical solution:

$$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})) = \frac{1}{2} \sum_{j=1}^{D_z} (\mu_{j,\phi}^2(\mathbf{x}) + \sigma_{j,\phi}^2(\mathbf{x}) - \log(\sigma_{j,\phi}^2(\mathbf{x})) - 1) \quad (18)$$

Derivation sketch: The general KL divergence between $Q = \mathcal{N}(\boldsymbol{\mu}_Q, \boldsymbol{\Sigma}_Q)$ and $P = \mathcal{N}(\boldsymbol{\mu}_P, \boldsymbol{\Sigma}_P)$ is $\frac{1}{2} \left(\text{Tr}(\boldsymbol{\Sigma}_P^{-1} \boldsymbol{\Sigma}_Q) + (\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q)^T \boldsymbol{\Sigma}_P^{-1} (\boldsymbol{\mu}_P - \boldsymbol{\mu}_Q) - D_z + \log \frac{|\boldsymbol{\Sigma}_P|}{|\boldsymbol{\Sigma}_Q|} \right)$. Plugging in $\boldsymbol{\mu}_Q = \boldsymbol{\mu}_\phi(\mathbf{x})$, $\boldsymbol{\Sigma}_Q = \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))$, $\boldsymbol{\mu}_P = \mathbf{0}$, and $\boldsymbol{\Sigma}_P = \mathbf{I}$ leads to the simplified formula above.

5.2 Reconstruction Term

This term measures how well the decoder can reconstruct the original input $\mathbf{x}$ given a latent code $\mathbf{z}$ sampled from the encoder's distribution for that $\mathbf{x}$. The exact form depends on the choice of $p_\theta(\mathbf{x}|\mathbf{z})$.

5.2.1 Case 1: Binary Data (e.g., MNIST)

Assume each pixel x_d (where d indexes the dimension/pixel) is a Bernoulli random variable. The decoder network (parameterized by θ) takes $\mathbf{z}$ and outputs a vector $\mathbf{p}_\theta(\mathbf{z})$, where each element $p_{d,\theta}(\mathbf{z})$ is the probability that pixel x_d is 1.

$$p_\theta(\mathbf{x}|\mathbf{z}) = \prod_{d=1}^{D_x} p_{d,\theta}(\mathbf{z})^{x_d} (1 - p_{d,\theta}(\mathbf{z}))^{1-x_d} \quad (19)$$

where D_x is the dimensionality of $\mathbf{x}$. The log-likelihood is:

$$\log p_\theta(\mathbf{x}|\mathbf{z}) = \sum_{d=1}^{D_x} [x_d \log p_{d,\theta}(\mathbf{z}) + (1 - x_d) \log(1 - p_{d,\theta}(\mathbf{z}))] \quad (20)$$

This is the negative binary cross-entropy between the input $\mathbf{x}$ and the reconstructed probabilities $\mathbf{p}_\theta(\mathbf{z})$. The reconstruction term becomes:

$$\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[\sum_{d=1}^{D_x} (x_d \log p_{d,\theta}(\mathbf{z}) + (1 - x_d) \log(1 - p_{d,\theta}(\mathbf{z}))) \right] \quad (21)$$

5.2.2 Case 2: Real-valued Data (e.g., Natural Images)

Assume each dimension x_d is Gaussian. The decoder network takes $\mathbf{z}$ and outputs the mean of the Gaussian $\boldsymbol{\mu}_\theta(\mathbf{z})$. We often assume a fixed variance, say $\sigma^2 \mathbf{I}$, for simplicity (e.g., $\sigma^2 = 1$), or the decoder could also output the variance. Assuming fixed variance σ^2 :

$$p_\theta(\mathbf{x}|\mathbf{z}) = \prod_{d=1}^{D_x} \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x_d - \mu_{d,\theta}(\mathbf{z}))^2}{2\sigma^2} \right) \quad (22)$$

The log-likelihood is:

$$\log p_\theta(\mathbf{x}|\mathbf{z}) = \sum_{d=1}^{D_x} \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x_d - \mu_{d,\theta}(\mathbf{z}))^2}{2\sigma^2} \right] \quad (23)$$

$$= -\frac{D_x}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{d=1}^{D_x} (x_d - \mu_{d,\theta}(\mathbf{z}))^2 \quad (24)$$

$$= -\frac{D_x}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|^2 \quad (25)$$

This is proportional to the negative squared error between the input $\mathbf{x}$ and the reconstructed mean $\boldsymbol{\mu}_\theta(\mathbf{z})$. The reconstruction term becomes:

$$\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[-\frac{D_x}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|^2 \right] \quad (26)$$

Since the constant term doesn't affect optimization w.r.t θ, ϕ , maximizing this term is equivalent to minimizing the expected squared error: $\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|^2]$.

6 The Reparameterization Trick

There's a problem with the reconstruction term $\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\dots]$. We need to compute its gradient with respect to ϕ . However, the expectation is taken over a distribution q_ϕ whose parameters $(\boldsymbol{\mu}_\phi, \boldsymbol{\sigma}_\phi^2)$ depend on ϕ . Standard backpropagation cannot handle gradients through the sampling process $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$.

The reparameterization trick solves this. For distributions like the Gaussian, we can rewrite the sampling process. Instead of sampling $\mathbf{z}$ directly from $\mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x})))$, we:

1. Sample a noise vector $\boldsymbol{\epsilon}$ from a fixed, simple distribution, typically a standard normal: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. Transform this noise using the parameters from the encoder:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon} \quad (27)$$

where $\odot$ denotes element-wise multiplication, and $\boldsymbol{\sigma}_\phi(\mathbf{x})$ is the vector of standard deviations (square root of the variances $\boldsymbol{\sigma}_\phi^2(\mathbf{x})$). Note that the encoder usually outputs $\log \boldsymbol{\sigma}_\phi^2(\mathbf{x})$, so $\boldsymbol{\sigma}_\phi(\mathbf{x}) = \exp(0.5 \times \log \boldsymbol{\sigma}_\phi^2(\mathbf{x}))$.

Now, $\mathbf{z}$ still follows the desired distribution $q_\phi(\mathbf{z}|\mathbf{x})$, but the stochasticity comes from $\boldsymbol{\epsilon}$, which is independent of ϕ . The dependence on ϕ is now deterministic through $\boldsymbol{\mu}_\phi(\mathbf{x})$ and $\boldsymbol{\sigma}_\phi(\mathbf{x})$.

We can rewrite the reconstruction term using this trick:

$$\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} [\log p_\theta(\mathbf{x}|\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon})] \quad (28)$$

In practice, we approximate this expectation using Monte Carlo sampling, typically with just one sample ($L = 1$) of $\boldsymbol{\epsilon}$ per data point $\mathbf{x}$ per training step:

$$\log p_\theta(\mathbf{x}|\mathbf{z}^{(i,l)}) \quad \text{where} \quad \mathbf{z}^{(i,l)} = \boldsymbol{\mu}_\phi(\mathbf{x}^{(i)}) + \boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(l)} \quad \text{and} \quad \boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (29)$$

7 The Full VAE Objective (for Training)

Combining everything, the objective function we want to maximize (or minimize its negative) for a single data point $\mathbf{x}$, using the reparameterization trick (Eq. 27) and the analytical KL divergence (Eq. 18), is approximated as:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) \approx \log p_\theta(\mathbf{x}|\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}) - \frac{1}{2} \sum_{j=1}^{D_z} (\mu_{j,\phi}^2(\mathbf{x}) + \sigma_{j,\phi}^2(\mathbf{x}) - \log(\sigma_{j,\phi}^2(\mathbf{x})) - 1) \quad (30)$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is sampled once.

We typically minimize the *negative* ELBO averaged over a mini-batch of M data points:

$$\text{Loss} = -\frac{1}{M} \sum_{i=1}^M \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) \quad (31)$$

$$\approx \frac{1}{M} \sum_{i=1}^M \left[\underbrace{-\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})}_{\text{Reconstruction Loss}} + \underbrace{\frac{1}{2} \sum_{j=1}^{D_z} (\mu_{j,\phi}^2(\mathbf{x}^{(i)}) + \sigma_{j,\phi}^2(\mathbf{x}^{(i)}) - \log(\sigma_{j,\phi}^2(\mathbf{x}^{(i)})) - 1)}_{\text{KL Loss}} \right] \quad (32)$$

where $\mathbf{z}^{(i)} = \boldsymbol{\mu}_\phi(\mathbf{x}^{(i)}) + \boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(i)}$ with $\boldsymbol{\epsilon}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

The term $-\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})$ is the reconstruction loss (e.g., binary cross-entropy or mean squared error, depending on the data type and choice of p_θ).

8 Training Algorithm Summary

1. Initialize encoder parameters ϕ and decoder parameters θ .
2. Repeat until convergence:
 - (a) Sample a mini-batch of data $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(M)}\}$.
 - (b) For each $\mathbf{x}^{(i)}$ in the mini-batch:
 - i. Pass $\mathbf{x}^{(i)}$ through the encoder network to get $\boldsymbol{\mu}_\phi(\mathbf{x}^{(i)})$ and $\log \boldsymbol{\sigma}_\phi^2(\mathbf{x}^{(i)})$.
 - ii. Calculate $\boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) = \exp(0.5 \times \log \boldsymbol{\sigma}_\phi^2(\mathbf{x}^{(i)}))$.
 - iii. Sample $\boldsymbol{\epsilon}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
 - iv. Compute the latent variable sample: $\mathbf{z}^{(i)} = \boldsymbol{\mu}_\phi(\mathbf{x}^{(i)}) + \boldsymbol{\sigma}_\phi(\mathbf{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(i)}$.
 - v. Pass $\mathbf{z}^{(i)}$ through the decoder network to get the parameters of $p_\theta(\mathbf{x}|\mathbf{z}^{(i)})$ (e.g., $\mathbf{p}_\theta(\mathbf{z}^{(i)})$ for Bernoulli, or $\boldsymbol{\mu}_\theta(\mathbf{z}^{(i)})$ for Gaussian).
 - vi. Calculate the reconstruction loss term $-\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})$.
 - vii. Calculate the KL divergence term $D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}^{(i)}) || p(\mathbf{z}))$ using the analytical formula (Eq. 18) with $\boldsymbol{\mu}_\phi(\mathbf{x}^{(i)})$ and $\boldsymbol{\sigma}_\phi^2(\mathbf{x}^{(i)})$.
 - (c) Compute the average loss over the mini-batch (Negative ELBO, Eq. 32).
 - (d) Compute gradients of the loss with respect to ϕ and θ using backpropagation (enabled by the reparameterization trick).
 - (e) Update ϕ and θ using an optimizer (e.g., Adam, SGD).

9 Generating New Data

Once the VAE is trained, we can generate new data points $\mathbf{x}_{new}$ that resemble the training data:

1. Sample a latent vector $\mathbf{z}_{new}$ from the prior distribution: $\mathbf{z}_{new} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. Pass $\mathbf{z}_{new}$ through the trained decoder network to obtain the parameters of $p_{\theta}(\mathbf{x}|\mathbf{z}_{new})$.
3. Sample a new data point $\mathbf{x}_{new}$ from this distribution $p_{\theta}(\mathbf{x}|\mathbf{z}_{new})$. (For example, if p_{θ} is Gaussian with mean $\boldsymbol{\mu}_{\theta}(\mathbf{z}_{new})$ and fixed variance σ^2 , then $\mathbf{x}_{new} \sim \mathcal{N}(\boldsymbol{\mu}_{\theta}(\mathbf{z}_{new}), \sigma^2 \mathbf{I})$). Often, for image generation, people just use the mean output of the decoder (e.g., $\mathbf{p}_{\theta}(\mathbf{z}_{new})$ or $\boldsymbol{\mu}_{\theta}(\mathbf{z}_{new})$) directly as the generated sample, rather than sampling from the output distribution.

This detailed breakdown covers the core mathematical concepts and formulas behind the Variational Autoencoder.